



Análisis de Rendimiento de Aplicaciones Paralelas - 2026

Trabajo Práctico Nº 4 - Evaluación de Rendimiento de Aplicaciones Paralelas



Objetivos de la práctica

-  **Instrumentar aplicaciones** para realizar mediciones temporales
 -  **Evaluar el rendimiento** de aplicaciones paralelas: métricas, escalabilidad fuerte y débil
-



Ejercicios



1 Análisis de Rendimiento con OpenMP

Este ejercicio consiste en analizar el rendimiento de una aplicación implementada con el modelo de programación paralela de **memoria compartida OpenMP**.



Importante: OpenMP solo funciona en máquinas de memoria compartida; por lo tanto, su uso se restringe a un **solo nodo del clúster**.

El modelo de ejecución de OpenMP consiste en lanzar un **único proceso** que a su vez lanzará **múltiples hilos de ejecución**.

Programa a Evaluar: Conjuntos de Julia

El programa calcula "**conjuntos de Julia**". Los conjuntos de Julia son una familia de conjuntos fractales que se obtienen al estudiar el comportamiento de los números complejos al ser iterados por una función holomorfa.

Ejemplo de archivo generado:

Conjunto de Julia

Observaciones Importantes

-  **Compilación:** Compilar el programa OpenMP `julia.c` con el compilador GCC, utilizar la opción de OpenMP y la opción de optimización `-O3`: `bash $ gcc -fopenmp -O3 julia.c -o julia`
-  **Parámetros:** El programa tiene dos parámetros, **alto** y **ancho** de la imagen. Por ejemplo, si ejecutamos `./julia 10000 2000` la imagen tendrá **10.000 filas por 2.000 columnas**.
-  **Cómputo:** El cómputo requerido para procesar todas las celdas de la matriz es **proporcional al número de celdas**.
-  **Salida:** La salida del programa indica el **tiempo de ejecución**.

Script de Referencia

El siguiente script puede servir de referencia para automatizar la ejecución de los experimentos con diferentes parámetros:

```
height=2000
width=1000

listOfCores="1 $(seq 2 2 8)"

for cores in $listOfCores; do
    echo Experimento con cores: $cores
    export OMP_NUM_THREADS=$cores
    ./julia $height $width
    echo -----
done
```

📌 La variable de entorno `OMP_NUM_THREADS` es utilizada para indicar, al entorno de ejecución de OpenMP, el **número de hilos a utilizar**.

Ejercicio 1.i: Escalabilidad Fuerte

Se requiere evaluar el rendimiento de la aplicación mediante un **estudio de escalabilidad fuerte**.

📌 Configuración:

-  Cantidad de cores: **2, 4, 6, 8, 10, ..., y hasta el número total de cores del nodo seleccionado**
-  Problema: **alto 240.000 y ancho 2.000**

✅ Se requiere:

- **a)**  Escribir el script utilizado para la ejecución de los experimentos
 - **b)**  Graficar la curva del **Tiempo de ejecución**
 - **c)**  Graficar la curva del **Speedup** (en el gráfico indicar también el speedup ideal/lineal)
 - **d)**  Graficar la curva de la **Eficiencia Paralela**
 - **e)**  Describir los resultados
-

Ejercicio 1.ii: Escalabilidad Débil

Se requiere evaluar el rendimiento de la aplicación mediante un **estudio de escalabilidad débil**.

Configuración:

-  Cantidad de cores: **2, 4, 6, 8, 10, ..., y hasta el número total de cores del nodo seleccionado**
-  Problema: **alto 8.000 y ancho 2.000 por core**

Se requiere:

- **a)**  Escribir el script que incluya el **cálculo del tamaño del problema** (alto y ancho) dependiente del número de cores de cada experimento

 **Ayuda:** para simplificar el cálculo, dejar fijo el ancho e ir variando la altura

- **b)**  Graficar la curva del **Tiempo de ejecución**
- **c)**  Describir los resultados

Instrumentación de un Programa Serie

Instrumentación de un programa serie para la medición de tiempos

Se provee el programa `programa1.c`, que muestra un ejemplo en el que se mide el tiempo transcurrido entre dos puntos de un programa.

Tareas:

-  Completar el programa `programa2.c` con **mediciones de tiempo**
-  Tener en cuenta que se desean realizar experimentos con **ejecuciones cortas** pero cuyos resultados sean **representativos de ejecuciones muy largas**
-  Indicar cómo se podría estimar el tiempo para una ejecución con: **$n = 1.000.000$**

Cálculo de Aceleración

Para los **ejercicios 8 y 10 del TP3**, calcular la **aceleración** de los programas optimizados frente a los programas sin optimizar.

Se requiere:

- ⚡ Calcular la **aceleración** (speedup) de los programas optimizados
 - 📊 Comparar con los programas sin optimizar
-
-



Soluciones



Soluciones - Trabajo Práctico N° 4: Análisis de Rendimiento de Aplicaciones Paralelas - 2026



Este documento contiene las respuestas completas a los ejercicios planteados en el README.md del TP4.



Índice

1. Ejercicio 1.i - Escalabilidad Fuerte  Ver enunciado en README
2. Ejercicio 1.ii - Escalabilidad Débil  Ver enunciado en README
3. Ejercicio 2 - Instrumentación de Programa Serie  Ver enunciado en README



Contexto: Programa de Conjuntos de Julia

El programa `julia.c` calcula el **conjunto de Julia**, un fractal que se obtiene iterando una función holomorfa con números complejos.



Imagen generada por el programa:

Conjunto de Julia



Ejemplo de fractal de Julia generado con el programa compilado con OpenMP.



Más detalles en el README:  Programa a Evaluar: Conjuntos de Julia

Ejercicio 1.i - Escalabilidad Fuerte



Enunciado completo en:  Ejercicio 1.i: Escalabilidad Fuerte (README)

a) Script de Ejecución

```
#!/bin/bash

height=240000
width=2000

# Obtener el número total de cores del nodo
max_cores=$(nproc)

# Generar lista de cores: 2, 4, 6, 8, 10, ..., max_cores
listOfCores="1 2 4 6 8"
for ((c=10; c<=max_cores; c+=2)); do
    listOfCores="$listOfCores $c"
done

echo "====="
echo "  ESCALABILIDAD FUERTE"
echo "  Problema: ${height} x ${width}"
echo "====="

for cores in $listOfCores; do
    echo ""
    echo "  🧪 Experimento con cores: $cores"
    export OMP_NUM_THREADS=$cores
    ./julia $height $width
    echo "-----"
done

echo ""
echo "  ✅ Todos los experimentos completados"
```

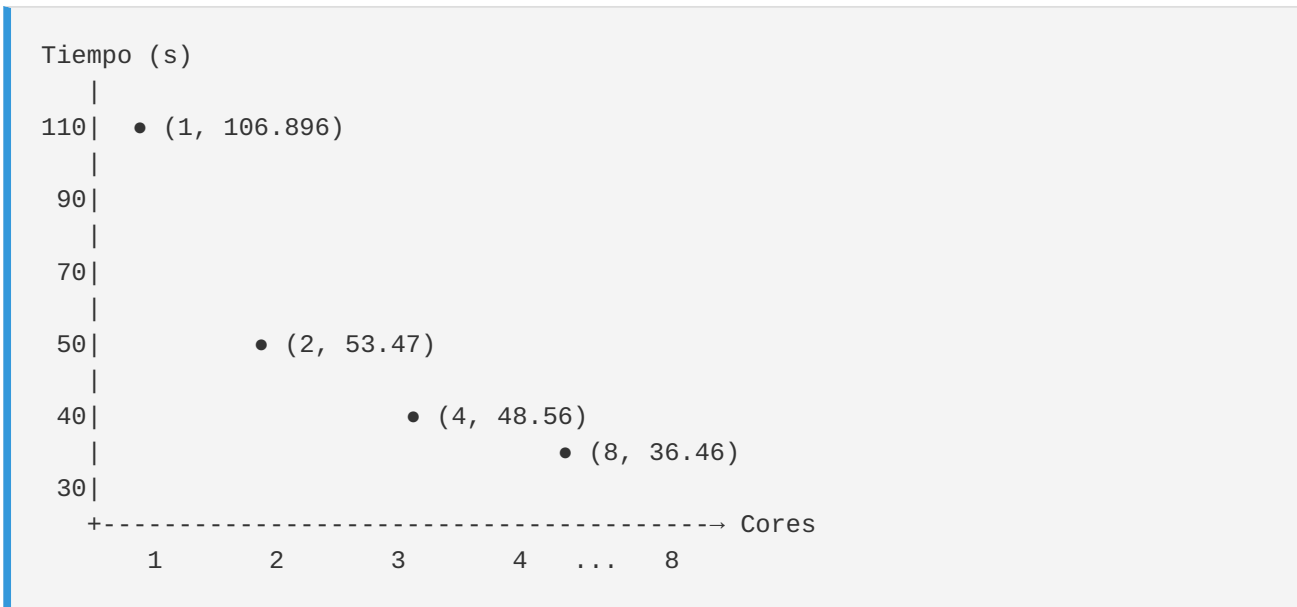


b) Curva de Tiempo de Ejecución

Cores (p)	Tiempo (Tp) [segundos]
1	106.896
2	53.470
4	48.563
8	36.460



Gráfico descriptivo: El tiempo de ejecución decrece a medida que aumentamos los cores, pero **no linealmente**. Se observa una reducción significativa al pasar de 1 a 2 cores (~50%), pero las mejoras se van reduciendo conforme se agregan más cores (ley de rendimientos decrecientes).



c) Curva de Speedup

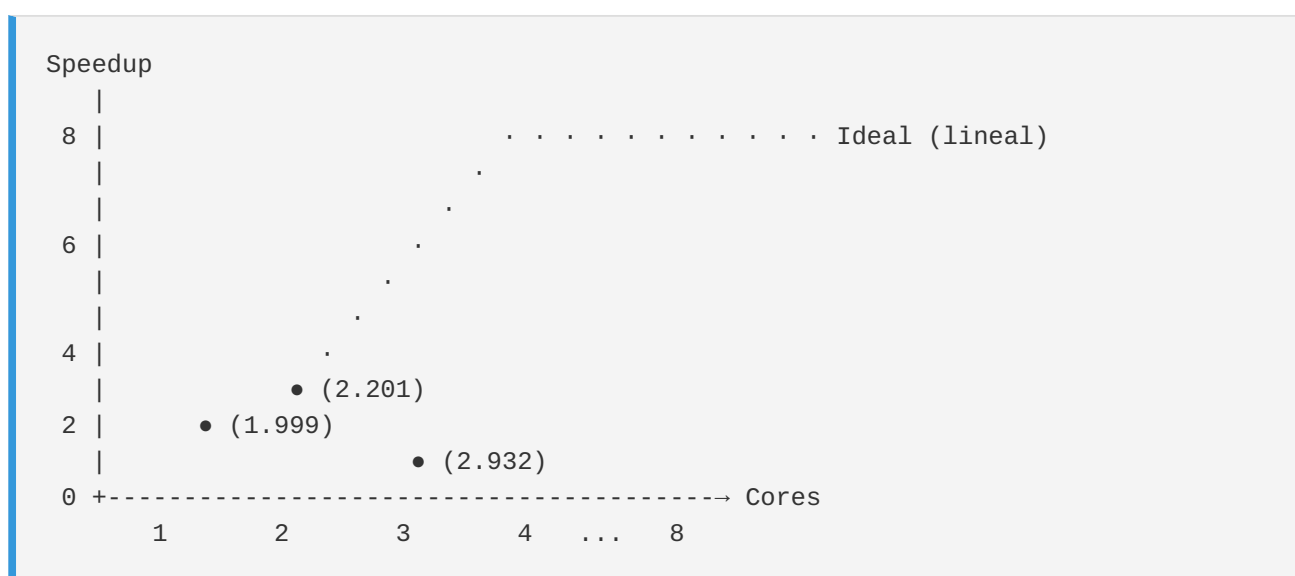


Cálculos del Speedup

La fórmula del speedup es: $S(p) = T(1) / T(p)$

Cores (p)	Speedup S(p)	Speedup Ideal
1	1.000	1
2	$106.896/53.470 = \mathbf{1.999}$	2
4	$106.896/48.563 = \mathbf{2.201}$	4
8	$106.896/36.460 = \mathbf{2.932}$	8

Gráfico descriptivo:



Observaciones del Speedup

⚠ El speedup se aleja del ideal: A medida que aumentamos la cantidad de cores, el valor del speedup se va **alejando progresivamente** del speedup ideal lineal. Mientras que con 2 cores se obtiene casi el speedup ideal (~ 2), con 8 cores apenas se alcanza ~ 2.93 cuando lo ideal sería 8. Esto indica que la paralelización **no escala eficientemente** con más cores.

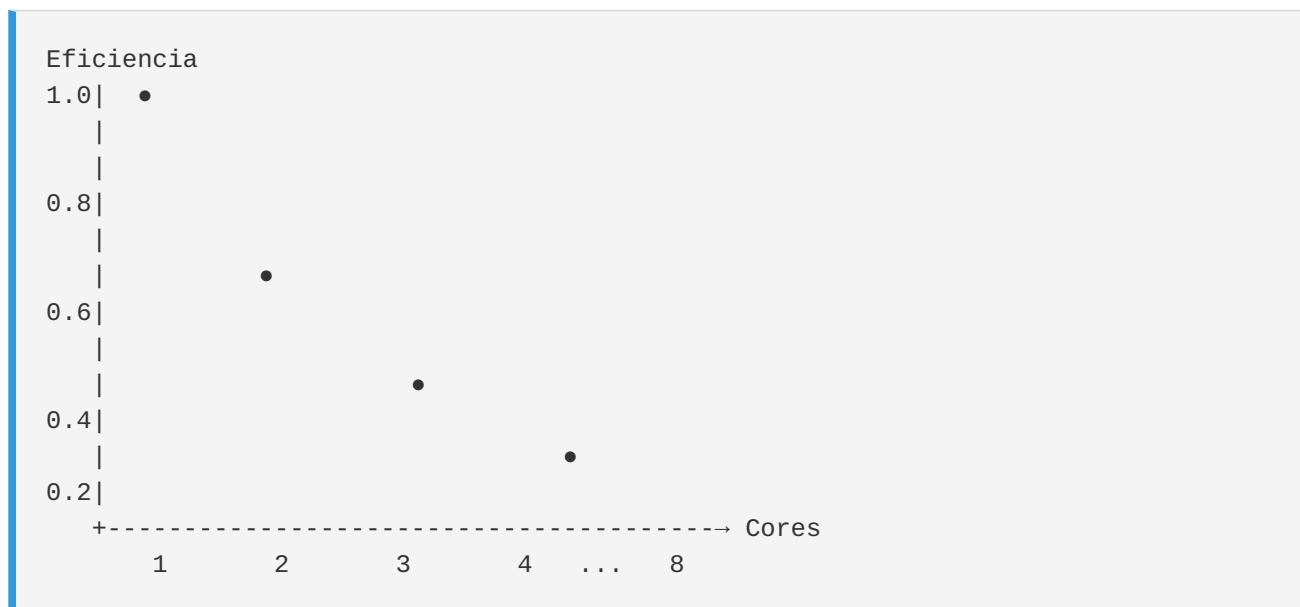
d) Curva de Eficiencia Paralela

Cálculos de Eficiencia

La fórmula de eficiencia es: $E(p) = S(p) / p$

Cores (p)	Eficiencia E(p)
1	$1/1 = 1.0000$
2	$1.999/2 = 0.9995$
4	$2.201/4 = 0.5503$
8	$2.932/8 = 0.3665$

Gráfico descriptivo:



Observaciones de Eficiencia Paralela

 **Decaimiento de eficiencia:** La eficiencia va **decayendo** a medida que se agregan más cores. Con 2 cores se mantiene casi perfecta (~99.95%), pero con 4

cores cae a ~55% y con 8 cores apenas ~36.6%. Esto indica que los cores adicionales están **subutilizados**, probablemente debido a:

-  **Overhead de sincronización** entre hilos
-  **Contención de memoria/cache** al acceder a datos compartidos
-  **Desbalance de carga** en la distribución del trabajo

Ejercicio 1.ii - Escalabilidad Débil

 **Enunciado completo en:**  [Ejercicio 1.ii: Escalabilidad Débil \(README\)](#)

a) Script con Cálculo del Tamaño del Problema

Derivación de la Fórmula

Para escalabilidad débil, el tamaño del problema crece proporcionalmente con los cores:

$$N_n = \sqrt{(n \times N_1^2)} / \text{ancho_fijo}$$

Donde:

$$N_1 = 8000 \times 2000 \quad (\text{tamaño base})$$

$$\text{ancho_fijo} = 2000$$

Simplificando:

$$N_n = \sqrt{n} \times 8000$$

Tabla de Tamaños de Problema

Cores (n)	Alto ($\sqrt{n} \times 8000$)	Ancho
1	8,000	2,000
2	11,313	2,000
4	16,000	2,000
8	22,627	2,000

```
#!/bin/bash

width=2000
base_height=8000

# Generar lista de cores
listOfCores="1 2 4 8"

echo "=====
echo "  ESCALABILIDAD DÉBIL"
echo "  Tamaño base: ${base_height} x ${width}"
echo "  Fórmula: alto =  $\sqrt{\text{cores}} \times \text{base\_height}$ "
echo "=====

for cores in $listOfCores; do
    # Calcular altura proporcional al número de cores
    height=$(echo "scale=0; sqrt($cores) * $base_height / 1" | bc)

    echo ""
    echo "  Experimento con cores: $cores | Problema: ${height} x ${width}"
    export OMP_NUM_THREADS=$cores
    ./julia $height $width
    echo "-----"
done

echo ""
echo "✅ Todos los experimentos completados"
```

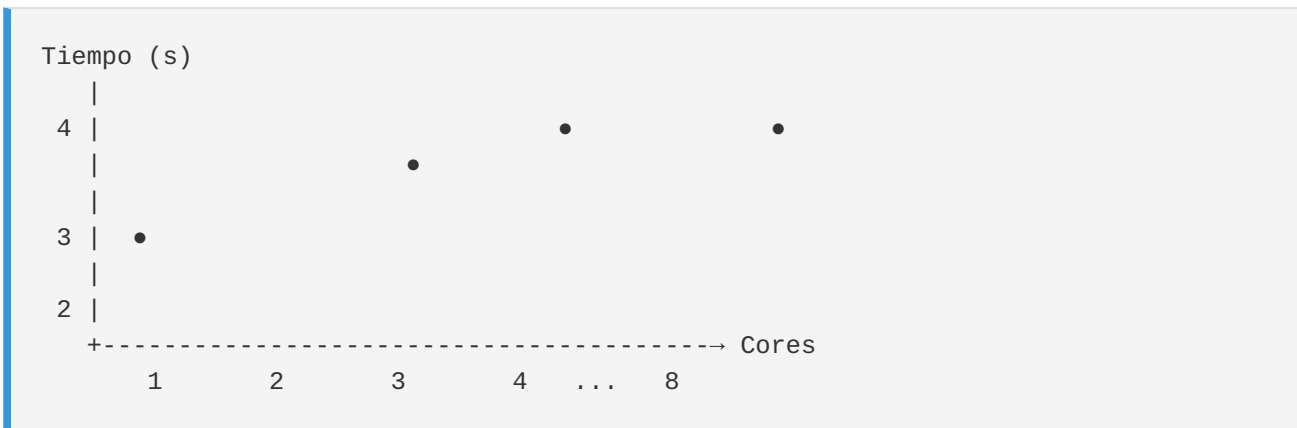


b) Curva de Tiempo de Ejecución

Cores (n)	Tamaño Problema	Tiempo (T _{n,p}) [segundos]
1	8,000 × 2,000	3.56
2	11,313 × 2,000	2.52
4	16,000 × 2,000	3.24
8	22,627 × 2,000	3.43



Gráfico descriptivo:



En escalabilidad débil ideal, el tiempo debería permanecer **constante** al aumentar proporcionalmente el problema con los recursos. Los tiempos se mantienen relativamente estables (~3-3.5s), lo cual es un buen indicador.



c) Descripción de Resultados



Cálculo de Eficiencia en Escalabilidad Débil

Fórmula: $E_{n,p} = T(1,1) / T(n,p)$

Cores (n)	Eficiencia $E_{n,p}$
1	$3.56/3.56 = \mathbf{1.000}$
2	$3.56/2.52 = \mathbf{1.413}$
4	$3.56/3.24 = \mathbf{1.099}$
8	$3.56/3.43 = \mathbf{1.038}$



Conclusiones

🎯 Eficiencia superior a 1 (Speedup Superlineal):

Observamos que la eficiencia está **por encima de 1** en varios casos, lo que indica un **speedup superlineal** ($S_{n,p} > p$). Esto se debe principalmente a:

- 🗄️ **Mayor memoria caché disponible:** Al usar más cores, hay más memoria caché rápida disponible en aggregate, haciendo que el programa acceda a los datos más rápidamente.
- 🧠 **Mejor localidad de datos:** La distribución de datos en cachés L1/L2 de cada core mejora el rendimiento.

⚠️ Consideraciones sobre memoria:

Se lograron realizar las mediciones exitosamente porque los valores de problema eran manejables por la memoria disponible. Sin embargo, si se extrapolan a valores mucho más grandes, **no sería posible** realizar la medición con problemas más grandes debido a problemas de **segmentación de memoria**.

🔧 En este caso, se contó con **10 GiB disponibles**, por lo que a partir de un problema de esa magnitud, el test de eficiencia sería **imposible a nivel práctico**.

Ejercicio 2 - Instrumentación de Programa Serie

 **Enunciado completo en:** [2](#) Instrumentación de un Programa Serie (README)

 **Programa Completado: programa2.c**


```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

#define X 1000

void leerDatos() {
    sleep(1); // simula la lectura de datos
}

void imprimirDatos() {
    sleep(1); // simula la escritura de datos
}

int main(int argc, char *argv[])
{
    int w, n, i, j, k, r, **a, **b, **c, **aux;
    struct timespec inicio, fin;
    double elapsed;

    if (argc != 2){
        printf("Argumento requerido: iteraciones\n");
        exit(1);
    }

    n = atoi(argv[1]);

    // Reserva de memoria
    a = malloc(X * X * sizeof(int*));
    b = malloc(X * X * sizeof(int*));
    c = malloc(X * X * sizeof(int*));
    for (i = 0; i < X; i++) {
        a[i] = malloc(X * sizeof(int));
        b[i] = malloc(X * sizeof(int));
        c[i] = malloc(X * sizeof(int));
    }

    // Inicialización de matrices
    leerDatos();

    // 🕒 Inicio de medición del kernel
    clock_gettime(CLOCK_MONOTONIC, &inicio);

    // Kernel de la aplicación
    for (w = 0; w < n; w++) {
        for(i = 0; i < X; ++i)
            for(k = 0; k < X; ++k) {
                r = a[i][k];
            }
    }
}

```

```

        for(j = 0; j < X; ++j)
            c[i][j] += r * b[k][j];
    }
    aux = a;
    a = c;
    c = aux;
}

// 🕒 Fin de medición del kernel
clock_gettime(CLOCK_MONOTONIC, &fin);
elapsed = (fin.tv_sec - inicio.tv_sec) + (fin.tv_nsec - inicio.tv_nsec) / 1e9;

printf("🕒 Tiempo del kernel (%d iteraciones): %.4f segundos\n", n, elapsed);
printf("📊 Tiempo por iteración: %.4f segundos\n", elapsed / n);

// Finalización
imprimirDatos();

// Liberar memoria
for (i = 0; i < X; i++) {
    free(a[i]);
    free(b[i]);
    free(c[i]);
}
free(a);
free(b);
free(c);

return 0;
}

```

🎯 Estimación del Tiempo para $n = 1.000.000$

📊 Datos Experimentales

Iteraciones (n)	Tiempo Total
1	9.3s
2	18.6s
3	27.9s

Cálculo

El orden del algoritmo es **$O(n)$** → **$T(n) = n \times c$**

Donde **c** es el tiempo por iteración:

$$c \approx 27.9s / 3 = 9.3s \text{ por iteración}$$

Estimación para $n = 1.000.000$:

$$T(1.000.000) = 1.000.000 \times 9.3s = 9.300.000 \text{ segundos}$$

 **Equivalente a: ~107 días** 

 **Nota:** Esta estimación asume comportamiento lineal perfecto. En la práctica, factores como caché, paginación de memoria y otros overheads podrían afectar el resultado real.

Ejercicio 3 - Cálculo de Aceleración

 **Enunciado completo en:** [3 Cálculo de Aceleración \(README\)](#)

Ejercicio 8 (TP3)

Versión	Tiempo [segundos]
Original	45.895
Optimizado	3.251

Cálculo de Speedup

```
Aceleración = T_original / T_optimizado  
              = 45.895 / 3.251  
              = 14.117
```

Observaciones

 El programa optimizado es **14.117 veces más rápido** que el original.

Esta mejora significativa se debe al **mejor uso de la caché**, contribuyendo a la **localidad espacial y temporal** de los accesos a memoria. Las optimizaciones de reordenamiento de bucles y acceso secuencial a memoria reducen drásticamente los cache misses.

Ejercicio 10 (TP3)

Versión	Tiempo [segundos]
Original	1.558
Optimizado	0.348

Cálculo de Speedup

```
Aceleración = T_original / T_optimizado  
              = 1.558 / 0.348  
              = 4.477
```

Observaciones

 El programa optimizado es **4.477 veces más rápido** que el original.

La mejora se debe a: -  **Mejor uso de la caché:** Optimización de patrones de acceso a memoria -  **Instrucciones vectoriales (SIMD):** Uso de instrucciones que procesan múltiples datos en una sola operación, aprovechando las unidades vectoriales del procesador

Resumen General de Resultados

Ejercicio	Métrica Principal	Valor	Observación
1.i	Speedup (8 cores)	2.932	Lejos del ideal (8) 
1.i	Eficiencia (8 cores)	36.65%	Baja utilización 
1.ii	Eficiencia débil (8 cores)	1.038	Speedup superlineal 
2	Tiempo estimado ($n=10^6$)	~107 días	Algoritmo $O(n)$ 
3 (Ej8)	Aceleración	14.117x	Optimización de caché 
3 (Ej10)	Aceleración	4.477x	Caché + SIMD 

Conclusiones Finales

-  **Escalabilidad Fuerte:** El speedup y eficiencia disminuyen conforme se agregan cores debido a overhead de paralelización y contención de memoria compartida.
-  **Escalabilidad Débil:** Se observó speedup superlineal gracias al mayor ancho de banda de memoria caché disponible al usar más cores.

3. 🕒 **Instrumentación:** La medición precisa de tiempos es fundamental para evaluar rendimiento. Con algoritmos $O(n)$, se pueden extrapolar tiempos a problemas mayores.
 4. 🚀 **Optimización:** Mejorar la localidad de caché y usar instrucciones vectoriales puede lograr aceleraciones de hasta **14x** sin cambiar el algoritmo base.
-



Trabajo Práctico Nº 4 completado — Sistemas Paralelos — 2026 